Markdown block editor

Generated from doc/markdown_block_editor.md on 2026-11-02

The admin Markdown editor (`MarkdownField`) has three views: **Blocks** (the default), **Code** (the Ace source editor, formerly "Edit") and **Preview**. This document describes how the block view works, which files take part in it, and what to keep in mind when changing it.

What the user gets

- The text is split into blocks (paragraphs, headings, lists, tables, code, iobjects, shortcodes...), each rendered by the server exactly as on the web.
- A block is edited in place (double-click, Enter or the pencil button): its Markdown source appears in a textarea. Ctrl+Enter or clicking outside saves, Esc cancels. The formatting buttons of the toolbar (H1–H3, bold, italic, lists, link, image) and Ctrl+B/I/K work on the edited block.
- Blocks are added by the "+" buttons (or by clicking into empty space below the last block), deleted, moved by dragging their handle or by Alt+Up/Down, cut, copied and pasted (the "⋮" menu, Ctrl+X/C/V on a focused block).
- Shortcode blocks with a registered editor open it in a modal: `[tabs]` the tab designer, `[row]` the layout designer, `[linklist]` the link list inserter, `[div]` the div editor (class and style), an iobject `[#123 ...]` the object picker.
- Container shortcodes – `[row]`, `[tabs]`, `[div]` – show their content as nested lists of blocks, so a column, a tab or a div is edited block by block too.
- Undo/redo works across the views (it is Ace's undo history).

The core rule: the source in Ace is the only truth

The block view never keeps its own copy of the document. Every change goes like this:

1. the current source is taken from Ace and tokenized into blocks (`MDBlockTokenizer.tokenize()`),
2. the change is made on the blocks (a block's `raw` replaced, a block inserted/removed/moved),
3. the blocks are joined back (`MDBlockTokenizer.join()`) and the result is written into Ace by `MDBlockEditor.setSource()` – only the changed part is replaced, so Ace's undo history gets one minimal change,
4. bootstrap-markdown-editor copies Ace's content into the textarea (its own change handler), so form submission and the "unsaved changes" checker work as before,
5. the source is tokenized again and the blocks are re-rendered (`refresh()`).

The tokenizer is **lossless**: `join( tokenize( source ) ) === source` for any source. Untouched parts of a document are never reformatted, so saving a page without changes in the block view produces no diff.

Never write into the textarea or the DOM of blocks as a way of changing the document. Use `applyDoc()` (after changing `this.doc`) or `setSource()`.

Files

JavaScript (`public/admin/scripts/utils/`)

File	Role
`md_shortcodes.js`	`UTILS.MDShortcodes`: finds shortcode tags in a source (ignoring code), builds a tree of paired tags, parses params the same way as DrInk Markdown (`parseAttributes()`), builds an opening tag (`buildOpeningTag()`). No DOM, runs in Node.js too.
`md_block_tokenizer.js`	`UTILS.MDBlockTokenizer`: splits a source into blocks, including the tree of container shortcodes; `join()`, `blockRaw()`, `normalize()`, `reorder()`. No DOM, runs in Node.js.
`md_row_layout.js`	`UTILS.MDRowLayout`: widths of `[col]` columns per breakpoint from Bootstrap classes and back; parses and rebuilds a `[row]` keeping content of the columns. No DOM, runs in Node.js.
`md_block_editor.js`	`UTILS.MDBlockEditor`: the block view itself (rendering, editing, menus, drag & drop, clipboard). One instance per MD editor (`container.mdBlockEditor`).
`md_editor_inserts.js`	`UTILS.MDEditorInserts`: registry of things insertable into editors (designers in modals). Fills the "Insert" dropdown, the "Add block" menu and finds the editor of a block.
`tab_designer.js`, `layout_designer.js`, `linklist_inserter.js`, `iobject_inserter.js`, `div_editor.js`	Designers registered in `MDEditorInserts`; each can insert a new thing and edit an existing one.
`fa_iconpicker.js`	Icon picker; **not migrated** to the registry yet – it still copies the code to the clipboard.
`md_editor_toolbar_helper.js`	Adds buttons, dropdowns and the view mode button ("Blocks") to toolbars of bootstrap-markdown-editor.
`md_editor_resizer.js`	Remembers editor heights in sessionStorage; the height is kept on `.md-editor` and mirrored to `.md-blocks`.
`preview_mode_toggle.js`	Desktop / tablet / phone width of the preview (used by the block view too).

Initialization is in `public/admin/scripts/application.js` (`initializeEditors()`): bootstrap-markdown-editor first, then the "Insert" dropdown, `UTILS.MDBlockEditor.init()`, `new UTILS.MDEditorResizer()` (must come after the block views exist), then the designers. All of it runs again after a form is replaced by AJAX (`edit_form_replaced`), so every `init()`/constructor is idempotent (singletons, "skip if already there").

PHP

File	Role
`app/fields/markdown_field.php`	Adds `data-block_shortcodes` (JSON list of block shortcodes known to DrInk Markdown, `MarkdownField::GetBlockShortcodes()`) to the textarea; the tokenizer needs it to keep a block shortcode together.

File	Role
`app/controllers/api/markdown_controller.php`	`transform` (one source, the original preview) and `transform_batch` (a JSON array of sources → a JSON array of HTML, max. 500 sources × 50 000 chars, `app/forms/api/markdown/transform_batch_form.php`). Both accept `editor_preview=1` and `base_href`. Admin only.
`app/helpers/function.admin_menu.php`	Returns nothing while `$ATK14_GLOBAL->getValue("admin_menu_disabled")` is set; the markdown controller sets it for `editor_preview`, so iobjects and shortcodes render without admin dropdowns.
`app/controllers/api/iobjects_controller.php`	`api/iobjects/?table_name=pages&record_id=2` – objects linked to a record with insert marks and preview images, for the object picker. Admin only.
`app/controllers/admin/iobjects_controller.php`	`edit` redirects to the editing of the iobject in its own controller (`pictures/edit`, `videos/edit`...); the "Edit object" link of iobject blocks uses it.
`app/helpers/block.drink_shortcode__tabs.php`	Tab names are parsed by `MarkdownShortcodesPostfilter::parseParams()`, so escaped (`name="A \"B\""`) or single quoted names work.

Templates and styles

- `app/views/admin/shared/layout/_md_block_editor_texts.tpl` – UI strings of the block view (`window.mdBlockEditorTexts`), rendered in the admin layout. English fallbacks are in `MDBlockEditor.texts()`.
- `app/views/admin/shared/_editor_modals.tpl` and `editor_modals/*` – modals of the designers, each with its own `window.*Texts`. The object picker needs the edited record: edit templates render `{render partial="shared/editor_modals" iobjects_for=$page}` (`application/edit.tpl` does it only for models with `has_iobjects`).
- `public/admin/styles/_md_block_editor.scss` – the block view; `_md_preview.scss` – the rendered content (frontend styles of iobjects, shortcodes... imported under `.md-preview, .md-blocks`); `_md_preview_grid.scss` – Bootstrap grid emulated by container queries (the preview width, not the window width, decides); `_iobject_inserter.scss`, `_tabdesigner.scss`, `_layout_designer.scss` – modals.

Tests

- `node test/js/run.js` – tests of `md_shortcodes.js`, `md_block_tokenizer.js`, `md_row_layout.js` (no dependencies, Node.js 9+).
- `./test/js/export_markdown_sources | node test/js/check_tokenizer_roundtrip.js [--verbose]` – runs the tokenizer on all page/article texts in the database and reports any source not surviving `tokenize()` + `join()`. Run it on production data (`ATK14_ENV=production ./test/js/export_markdown_sources > sources.json`) after changing the tokenizer.
- PHP: `test/controllers/api/tc_markdown.php`, `tc_iobjects.php`, `test/controllers/admin/tc_iobjects.php`, `test/fields/tc_markdown_field.php`, `test/helpers/tc_drink_shortcode_tabs.php`.

How the tokenizer works

`tokenize( source, { blockShortcodes, containers } )` returns `{ leading, blocks, trailing, definitions }`. Every block has `type`, `raw` (its exact source), `separator` (whitespace up to the next block), `line` (0-based, in the whole source) and type specific fields (`level`, `name`, `attrs`, `id`, `unclosed`...).

- Boundaries are found line by line: blank lines end a block, except inside fenced code, inside a block shortcode (tags are counted, nested ones too) or an HTML block (until its element is closed); a loose list continues after blank lines; a heading or a block shortcode starts a new block even without a blank line.
- An iobject block is a paragraph consisting only of `[#123 ...]` – DrInk Markdown renders an iobject only then.
- `definitions` are link reference definitions (`[1]: http://...`) from the whole document; the editor appends them to every rendered block, because a block rendered alone wouldn't see them.

Containers. `containers` (default `MDBlockTokenizer.defaultContainers`) says which shortcodes are split further:

```
{ row: { children: "col" }, tabs: { children: "tab" }, div: { content: true, nestable: true }
}
```

A container block gets `block.container = { head, children: [ { name, attrs, open, doc, close, after } ], tail }`; each child's content is tokenized recursively into `doc` (marked `doc.nested`). `children` containers consist of the given child shortcodes only (whitespace between them is allowed, anything else makes the shortcode a plain block); `content` containers have their own content as the only "child". `nestable` allows a container inside a child of its own kind in the editor (`[div]` in `[div]`, but not `[row]` in a column).

`join()` builds the `raw` of a container from its parts (`blockRaw()`), so a change in a nested doc reaches the source. **When a container's `raw` is replaced as a whole, delete `block.container`**, otherwise `blockRaw()` rebuilds the old content.

`normalize( doc, indexes )` makes sure the given blocks are followed by a blank line (needed after inserting, removing or moving blocks; a paragraph followed by another one without a blank line would merge). Pass only the indexes around the change, so the rest of the document stays as it was. For nested docs it also puts multi-line content on its own lines (`[col]Text[/col]` → `[col]\nText\n\nMore\n[/col]`). `reorder( doc, order )` moves blocks and normalizes only new neighbours.

How the block view works

Paths. Blocks are addressed by paths: `"3"` is the 4th top-level block, `"3/1/0"` the 1st block of the 2nd child of block 3. A list is addressed by the path of its container and the child index (`"3/1"`), the top-level list is `""`. `resolve( path )` and `resolveList( list )` find them in `this.doc`; paths change after every re-tokenization, never keep them across `applyDoc()`.

Rendering.

- Every block is rendered by `api/markdown/transform_batch` (one request for all blocks missing in the cache). `MDBlockEditor.renderCache` (shared by all editors in the page) maps `base_href + source` to HTML.
- A container is rendered as a *skeleton*: its children's content is replaced by `<div class="md-slot" data-slot="N"></div>`, the server renders the real wrapper HTML (grid classes, tab navigation...), and the nested lists of blocks are put into the slots (`mountLists()`). This keeps the look identical to the web. A new container shortcode is suitable only if its template outputs the child content as it is (the slot must survive) – check it in `./scripts/console` with `smarty_modifier_markdown()`.
- Block elements are reused when their source didn't change (`elementFor()` keyed by the rendered source), so interactive components keep their state (an active tab) and media aren't reloaded. `placeElements()` moves as few nodes as possible – moving a node reloads its iframes.
- `adaptLayouts()`: a nested list put into a flex/grid element (e.g. `[div class="card-deck"]`) would break the layout, so the list takes over the parent's flex/grid properties and every block takes over the flex properties of its rendered content. It runs after every render and after a preview width change.

Editing. `editBlock( path )` opens a registered editor (`MDEditorInserts.editorFor( block )`) or the inline textarea (`startEdit()`). The edit is committed on blur, but not when the focus goes to the editor's toolbar (the "Insert" dropdown) or while a modal of an inserter is open (`editing.suspended`). A click that only finishes an edit or closes a menu doesn't start a new block (`busyAtMousedown`).

Integration with bootstrap-markdown-editor. The library is not modified. Its toolbar buttons have jQuery handlers bound at init; the block view adds capturing listeners on the toolbar to redirect formatting buttons to the edited block and to keep the focus in the textarea. Its "Edit" button is renamed to "Code" in `MDBlockEditor.init()`. Ace doesn't redraw changes made while hidden – `redrawAce()` runs after switching to Code.

Drag & drop. Every list has its own SortableJS instance with a group per editor (blocks can't be dragged into another field). `forceFallback` keeps the dragged clone inside `.md-blocks`, so it is styled as content. `onMove` forbids a block inside itself and a non-nestable container inside a child of its own kind; `canPasteInto()` applies the same rule to Paste.

Clipboard. Cut/Copy store the block's source (a container with all its content) in `localStorage` (`md_block_editor_clipboard`), so it can be pasted on another page, and in the system clipboard as text. Paste reads only the stored source (reading the system clipboard would need a permission prompt).

The registry of inserts (designers)

```
UTILS.MDEditorInserts.register( {
  name: "tabs", text: "...", title: "Tabs",
  modal: "#tabdesigner_modal",
  inline: false,                 // true: a piece of text, not offered in "Add block"
  shortcode: "tabs",             // or blockType: "iobject" – which blocks it edits
  parse: raw => model || null,
  open: ( { raw, save } ) => { ... save( code ); },
} );
```

`run()` shows the modal, calls `open()` and waits for the modal to close: `save( code )` delivers the code, closing without saving cancels. The same `open()` serves inserting (`raw` is null) and editing (`raw` is the block's source).

Rules for a designer:

- `parse()` must return `null` for anything it can't represent without losing something (text between tabs, unknown structure...); the block is then edited as source instead.
- Keep what the form doesn't show (other attributes, classes, content of columns).
- When nothing was changed in the form, save the original `raw` exactly, otherwise opening and saving a block would reformat it.
- DrInk Markdown params can't contain `]`; double quotes are escaped as `\"` (`MDShortcodes.buildOpeningTag()`). Server-side helpers reading params of nested tags must use `parseParams()`, not their own regex (see the tabs helper).
- Modals are shown by `MDEditorInserts.showModal()`; it works with Bootstrap 4 (jQuery) and 5. `window.bootstrapVersion` used in older code is never set, don't rely on it.

`[row]` specifics (`md_row_layout.js`): a `[col]` without `defaultclasses=0` gets default classes depending on the **number of columns** (`_col.tpl`: `col-md-{12/n}`, special cases for 4 and 6), so a column kept as `[col]` changes its width when columns are added – `serialize()` checks that with the new count. Removed columns' content is moved to the last column.

Things to watch out for

- **Losslessness.** Any change of the tokenizer must keep `join( tokenize( x ) ) === x`; add a test and run the database check. Don't "clean up" whitespace of blocks the user didn't touch.
- **Rendering blocks in isolation.** A block doesn't see the rest of the document: reference definitions are appended for that reason; numbered list continuation or heading ids may differ from the full render. Preview renders the whole document and stays the reference.
- **CSS of the content.** Frontend styles apply under `.md-preview, .md-blocks`. Rules aimed at direct children of shortcode wrappers (`.col--shortcode > .iobject`, flex/grid layouts) don't match in the block view, where the content is wrapped by `.md-blocks-list > .md-block > .md-block__content`; add an equivalent to the dedicated section at the end of `_md_block_editor.scss` instead of changing the frontend styles.
- **No layout shifts.** The block toolbar sits in a zero-height sticky bar (`.md-block__bar`) and is positioned in it, the "+" is absolutely positioned; anything shown on hover must stay out of the flow, otherwise margins of the block's content stop collapsing and the content jumps.
- **Pointer events.** Iframes in blocks have `pointer-events: none` (hover and double-click would go to the embedded page); the dragged clone and everything in it have `pointer-events: none` (content like `.row--shortcode` turning them back on would hide the drop target from SortableJS).
- **Dark mode.** Content is rendered on white even in the dark mode, so inputs inside blocks are forced light, and tables of the content get their light Bootstrap colors back (`_md_preview.scss`; `_dark_mode.scss` darkens every `.table`). Excluding content from dark mode rules by `:not(.md-preview .table)` doesn't work: node-sass silently drops a `:not()` with a complex selector.
- **Heights.** The editor height is kept on `.md-editor` (stored by `md_editor_resizer.js`); a hidden editor has no `offsetHeight`, use its style height.

- **Performance.** Every MD field renders its blocks right after page load (one batch request per field). The single-threaded `./scripts/server` serves them one by one; on a real server they run in parallel.
- **Translations.** New UI strings are in `{t}` in the templates; Czech translations in `locale/` haven't been added yet.

Building

Admin scripts are concatenated in the order of `applicationScripts` in `gulpfile-admin.js` – a new file must come after its dependencies (`md_shortcodes.js` before the tokenizer, `md_editor_inserts.js` before the designers...). `gulp serve-admin` reads the list only when it starts: restart it after changing `gulpfile-admin.js`, otherwise new files are left out of the bundle and the admin fails on an undefined class. The build has been run with Node.js 10 (Gulp 3); with Node.js 9 it fails in `postcss`.

Known limitations and ideas

- Paste only pastes blocks copied in the block view, not text from the system clipboard.
- New blocks can be inserted after a block or at the end of a list, not before the first block (dragging solves it).
- A column made multi-line by adding a block stays multi-line after the block is removed.
- A block can't be dropped into a hidden tab; switch to the tab first.
- The icon picker isn't in the registry yet (it should be `inline: true`, inserted into the edited block).
- Other shortcodes could become containers (`[collapse]`, `[collapse_content]`) by adding them to `defaultContainers` after checking their skeleton renders with slots.